

# Monte Carlo & Binomial Tree: Concepts and Code Walkthrough

Companion note to Stage 2 — how each method works, why it's relevant to option pricing, and what each function in `monte_carlo.py` and `binomial_tree.py` actually does.

---

## Part 1: The Concepts

### 1.1 Why cross-validate Black-Scholes at all?

Stage 1 implemented Black-Scholes as a single closed-form formula. The strongest way to trust that implementation is not just to check it against itself (edge cases, put-call parity), but to price the exact same option using a completely different mathematical approach that doesn't use the formula at all. If two unrelated methods agree, that's much stronger evidence of correctness than one method passing its own internal checks. Monte Carlo and the binomial tree are the two standard ways to do this.

### 1.2 Monte Carlo simulation

Instead of solving an equation, Monte Carlo simulates the future directly. Under the same risk-neutral random-walk assumption that Black-Scholes itself is built on, the stock's price at expiry can be simulated as:

$$S_T = S_0 \exp \left( \left( r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right)$$

where  $Z$  is a single random draw from a standard normal distribution. Drawing a fresh random  $Z$  generates one possible future for the stock; drawing many  $Z$ 's generates many possible futures — this is exactly what "geometric Brownian motion" means in practice: log-returns that are normally distributed. For each simulated future price, you can calculate what the option would pay off (e.g.,  $\max(S_T - K, 0)$  for a call), then average across every simulation and discount back to today:

$$\hat{C} = \frac{1}{N} \sum_{i=1}^N e^{-rT} \max(S_T^{(i)} - K, 0)$$

This is a direct estimate of the option's expected discounted payoff — which is, by construction, exactly what an option's price *is* under the no-arbitrage pricing theory from Stage 1. As the number of simulations  $N$  grows, the law of large numbers guarantees this average converges to the true expected value, which is the Black-Scholes price. **Relevance to option pricing:** Monte Carlo is the standard method in industry whenever a payoff is too complex for a closed-form solution to exist (path-dependent payoffs, multiple underlying assets, exotic structures) — you give up an exact formula but gain the ability to price almost anything you can simulate.

### 1.3 The binomial tree

The binomial tree takes the opposite approach: rather than simulating many random continuous paths, it simplifies time itself. It breaks the time to expiry into  $n$  discrete steps, each of length  $dt$ , and assumes that at each step the stock can only do one of two things: move up by a factor  $u$ , or down by a factor  $d$ :

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = \frac{1}{u}$$

and assigns a risk-neutral probability  $p$  to the up-move, chosen so the tree is consistent with earning the risk-free rate on average:

$$p = \frac{e^{r\Delta t} - d}{u - d}$$

This branching creates a "tree" of every possible stock price path. At the final layer (expiry), the option's value is simply its payoff — no uncertainty left. The algorithm then works *backward* one layer at a time: at every earlier node, the option's value is the discounted, probability-weighted average of its two possible next values:

$$V_{\text{node}} = e^{-r\Delta t} (p V_{\text{up}} + (1 - p) V_{\text{down}})$$

Repeating this back to today gives the option's price. **Relevance to option pricing:** as  $n$  grows large, this discrete up/down process becomes indistinguishable from the continuous randomness Black-Scholes assumes — which is why the tree's price converges to the Black-Scholes price. More importantly in practice, because the tree checks in at every single step, it can also handle **American-style early exercise** (Section 2 of the Stage 1 note) simply by comparing, at every node, "what is this worth if I hold it" against "what is this worth if I exercise it right now" — something the closed-form Black-Scholes formula fundamentally cannot do.

#### 1.4 The two methods, side by side

|                                  | Monte Carlo                                    | Binomial Tree                            |
|----------------------------------|------------------------------------------------|------------------------------------------|
| <b>Core idea</b>                 | Simulate many random futures directly          | Discretize time into up/down steps       |
| <b>Converges by</b>              | More simulations (law of large numbers)        | More time steps (discrete to continuous) |
| <b>Handles American options?</b> | Not naturally                                  | Yes — its main practical advantage       |
| <b>Used in practice for</b>      | Exotic/path-dependent payoffs, multiple assets | American options, simple structures      |

## Part 2: Code Walkthrough — monte\_carlo.py

Two functions. The first generates the random future prices; the second uses them to produce a price estimate.

**simulate\_terminal\_prices(S, T, r, sigma, n\_simulations, seed)**

Generates  $n_{\text{simulations}}$  random possible stock prices at expiry, using the geometric Brownian motion formula from Section 1.2.

```
rng = np.random.default_rng(seed)
```

```
Z = rng.standard_normal(n_simulations)
S_T = S * np.exp((r - 0.5*sigma**2)*T + sigma*np.sqrt(T)*Z)
```

**rng.standard\_normal(n\_simulations)** draws `n_simulations` independent values from the standard normal distribution in one vectorized call — this is the `Z` from the formula, generated many times at once rather than in a slow Python loop. The next line applies the GBM formula to every one of those draws simultaneously (NumPy broadcasts the arithmetic across the whole array), producing an array of `n_simulations` different possible future stock prices. The seed parameter makes the randomness reproducible: the same seed always produces the same random draws, which matters for debugging and for fair before/after comparisons.

### **monte\_carlo\_price(S, K, T, r, sigma, n\_simulations, option\_type, seed)**

Uses the simulated prices above to actually estimate the option's price, and reports how precise that estimate is.

```
S_T = simulate_terminal_prices(S, T, r, sigma, n_simulations, seed)
payoffs = np.maximum(S_T - K, 0) # (or K - S_T for a put)
discounted_payoffs = np.exp(-r * T) * payoffs
price = discounted_payoffs.mean()
standard_error = discounted_payoffs.std(ddof=1) / np.sqrt(n_simulations)
```

For each simulated terminal price, **np.maximum(S\_T - K, 0)** computes what the option would actually pay off in that scenario — zero if the stock ended up below the strike, the difference otherwise. This is done for all simulations at once as an array operation. Each payoff is then discounted back to today's dollars (**np.exp(-r\*T)**), and **.mean()** averages across every simulation — this single line is the Monte Carlo estimator formula from Section 1.2, computed directly. The **standard\_error** line matters as much as the price itself: it's the estimate's own statistical uncertainty (standard deviation of the payoffs, divided by the square root of the sample size — the standard formula for the standard error of a mean). This is what lets you report a proper confidence interval, and what makes the convergence chart in Stage 2 show a shrinking band rather than just a single line.

## **Part 3: Code Walkthrough — binomial\_tree.py**

One function, doing three things in sequence: setting up the tree's parameters, computing the payoff at every final node, then working backward to today.

### **binomial\_tree\_price(S, K, T, r, sigma, n\_steps, option\_type, exercise\_style)**

#### **Step 1 — set up the tree's parameters:**

```
dt = T / n_steps
u = np.exp(sigma * np.sqrt(dt))
d = 1 / u
p = (np.exp(r * dt) - d) / (u - d)
discount = np.exp(-r * dt)
```

`dt` is the length of one time step. `u` and `d` are exactly the up/down factors from Section 1.3 — note `d` is defined as `1/u`, the standard Cox-Ross-Rubinstein convention that keeps the tree "recombining" (an up-then-down move lands on the same price as a down-then-up move, which is what keeps the tree's size manageable rather than exploding combinatorially). `p` is the risk-neutral probability formula from Section 1.3, computed once since it's the same at every node in a CRR tree. `discount` is the one-step discount factor, applied repeatedly as the algorithm works backward.

#### **Step 2 — the payoff at every final node:**

```
j = np.arange(n_steps + 1)
stock_at_expiry = S * (u ** j) * (d ** (n_steps - j))
values = np.maximum(stock_at_expiry - K, 0) # (or K - stock for a put)
```

After `n_steps` steps, there are exactly `n_steps+1` distinct final prices possible (from "down every time" to "up every time"), because the tree recombines. `j` indexes how many of those steps were up-moves; `stock_at_expiry` computes all `n_steps+1` final prices at once as a NumPy array. `values` then holds the option's payoff at each of

those final prices — this is the last layer of the tree, where the option's value is simply known, with no more uncertainty left.

### Step 3 — work backward to today:

```
for step in range(n_steps - 1, -1, -1):
    values = discount * (p*values[1:] + (1-p)*values[:-1])
    if exercise_style == "american":
        j = np.arange(step + 1)
        stock_at_step = S * (u**j) * (d**(step - j))
        immediate_exercise = np.maximum(stock_at_step - K, 0)
        values = np.maximum(values, immediate_exercise)
```

This loop is the heart of the method, implementing the backward-step formula from Section 1.3 directly. At each iteration, **values[1:]** and **values[:-1]** are each node's two possible "next step" values (an up-move and a down-move), so this one line computes the discounted, probability-weighted value at every node in the current layer simultaneously — the array shrinks by one element each pass through the loop, exactly mirroring the tree narrowing as you step back toward today. The **if exercise\_style == "american"** block is what gives the tree its key practical advantage over Black-Scholes: at every single step, it recomputes what the stock price would be at that point (**stock\_at\_step**), works out what exercising immediately would be worth (**immediate\_exercise**), and takes the larger of "keep holding" versus "exercise now" at every node. For a European option this block is simply skipped, since early exercise isn't allowed. After **n\_steps** iterations, exactly one value remains — **values[0]** — which is the option's price today.

---

*Both functions were validated in `test_convergence.py`: the binomial tree's price converges to Black-Scholes as `n_steps` grows, and Monte Carlo's price converges as `n_simulations` grows — confirming both implementations above are computing what this note describes.*